

程式碼實驗室：在 Node.js 中訓練 TensorFlow.js 模型

來源：<https://codelabs.developers.google.com/codelabs/tensorflowjs-nodejs-codelab?hl=zh-tw>

步驟數：9

在這個程式碼研究室中，您將瞭解如何在 Node.js 伺服器上使用 TensorFlow.js 建構及訓練棒球提案估算模型，並為用戶端提供指標。

目錄

1. [1. 簡介](#)
2. [2. 需求條件](#)
3. [3. 設定 Node.js 應用程式](#)
4. [4. 設定訓練和測試資料](#)
5. [5. 建立模型來分類投球類型](#)
6. [6. 在伺服器上訓練模型](#)
7. [7. 建立用戶端網頁並顯示程式碼](#)
8. [8. 執行應用程式](#)
9. [9. 您學到的內容](#)

步驟 1 · 約 3 分鐘

1. 簡介

在本程式碼實驗室中，您將瞭解如何建構 Node.js 網路伺服器，使用 [TensorFlow.js](#) (功能強大且彈性十足的 JavaScript 機器學習程式庫)，在伺服器端訓練及分類棒球的投球類型。您將建構網頁應用程式，訓練模型來預測音高感應器資料的音高類型，並從網頁用戶端叫用預測功能。本程式碼研究室的完整運作版本位於 [tfjs-examples](#) GitHub 存放區。

課程內容

- 如何安裝及設定 tensorflow.js npm 套件，以便搭配 Node.js 使用。
- 如何在 Node.js 環境中存取訓練和測試資料。
- 如何在 Node.js 伺服器中，使用 TensorFlow.js 訓練模型。
- 如何在用戶端/伺服器應用程式中部署訓練好的模型，以進行推論。

現在就開始吧！

步驟 2 · 約 2 分鐘

2. 需求條件

如要完成本程式碼研究室，您需要：

1. 使用新版 [Chrome](#) 或其他新式瀏覽器。
 2. 在本機電腦上執行的文字編輯器和指令終端機。
 3. 熟悉 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#) (或您偏好的瀏覽器開發人員工具)。
 4. 對類神經網路有高階概念性瞭解。如需簡介或複習，建議觀看 [3blue1brown 的這部影片](#)，或是 [Ashi Krishnan 的這部 JavaScript 深度學習影片](#)。
-

步驟 3 · 約 5 分鐘

3. 設定 Node.js 應用程式

安裝 Node.js 和 npm。如要瞭解支援的平台和依附元件，請參閱 [tfjs-node 安裝指南](#)。

為 Node.js 應用程式建立 ./baseball 目錄。將連結的 [package.json](#) 和 [webpack.config.js](#) 複製到這個目錄，設定 npm 套件依附元件 (包括 **@tensorflow/tfjs-node** npm 套件)。然後執行 npm install 安裝依附元件。



```
$ cd baseball
$ ls
package.json  webpack.config.js
$ npm install
...
$ ls
node_modules  package.json  package-lock.json  webpack.config.js
```

現在您已準備好編寫程式碼並訓練模型！

步驟 4 · 約 10 分鐘

4. 設定訓練和測試資料

您將使用下列連結中的訓練和測試資料做為 CSV 檔案。下載並探索這些檔案中的資料：

[pitch_type_training_data.csv](#)

[pitch_type_test_data.csv](#)

以下是一些訓練資料範例：

```
vx0,vy0,vz0,ax,ay,az,start_speed,left_handed_pitcher,pitch_code
7.69914900671662,-132.225686405648,-6.58357157666866,-22.5082591074995,28.3119270826735,-16.5
850095967027,91.1,0,0
6.68052308575228,-134.215511616881,-6.35565979491619,-19.6602769147989,26.7031848314466,-14.3
430602022656,92.4,0,0
2.56546504690782,-135.398673977074,-2.91657310799559,-14.7849950586111,27.8083916890792,-21.5
737737390901,93.1,0,0
```

共有八項輸入特徵，用於描述音高感應器資料：

- 球速 (vx0、vy0、vz0)
- 球體加速度 (ax、ay、az)
- 投球的起始速度
- 投手是否為左撇子

和一個輸出標籤：

- pitch_code，代表七種音高類型之一：`Fastball (2-seam)`、`Fastball (4-seam)`、`Fastball (sinker)`、`Fastball (cutter)`、`Slider`、`Changeup`、`Curveball`

目標是建構模型，根據投球感應器資料預測投球類型。

建立模型前，請先準備訓練和測試資料。在 `baseball/` 目錄中建立 `pitch_type.js` 檔案，然後將下列程式碼複製到檔案中。這段程式碼會使用 [tf.data.csv](#) API 載入訓練和測試資料。此外，這項作業也會使用最小值與最大值正規化比例，將資料正規化 (一律建議這麼做)。

```

const tf = require('@tensorflow/tfjs');

// util function to normalize a value between a given range.
function normalize(value, min, max) {
  if (min === undefined || max === undefined) {
    return value;
  }
  return (value - min) / (max - min);
}

// data can be loaded from URLs or local file paths when running in Node.js.
const TRAIN_DATA_PATH =
'https://storage.googleapis.com/mlb-pitch-data/pitch_type_training_data.csv';
const TEST_DATA_PATH = 'https://storage.googleapis.com/mlb-pitch-
data/pitch_type_test_data.csv';

// Constants from training data
const VX0_MIN = -18.885;
const VX0_MAX = 18.065;
const VY0_MIN = -152.463;
const VY0_MAX = -86.374;
const VZ0_MIN = -15.5146078412997;
const VZ0_MAX = 9.974;
const AX_MIN = -48.0287647107959;
const AX_MAX = 30.592;
const AY_MIN = 9.397;
const AY_MAX = 49.18;
const AZ_MIN = -49.339;
const AZ_MAX = 2.95522851438373;
const START_SPEED_MIN = 59;
const START_SPEED_MAX = 104.4;

const NUM_PITCH_CLASSES = 7;
const TRAINING_DATA_LENGTH = 7000;
const TEST_DATA_LENGTH = 700;

// Converts a row from the CSV into features and labels.
// Each feature field is normalized within training data constants
const csvTransform =
  ({xs, ys}) => {
    const values = [
      normalize(xs.vx0, VX0_MIN, VX0_MAX),
      normalize(xs.vy0, VY0_MIN, VY0_MAX),
      normalize(xs.vz0, VZ0_MIN, VZ0_MAX), normalize(xs.ax, AX_MIN, AX_MAX),
      normalize(xs.ay, AY_MIN, AY_MAX), normalize(xs.az, AZ_MIN, AZ_MAX),
      normalize(xs.start_speed, START_SPEED_MIN, START_SPEED_MAX),
      xs.left_handed_pitcher
    ];
    return {xs: values, ys: ys.pitch_code};
  }

const trainingData =

```

```
tf.data.csv(TRAIN_DATA_PATH, {columnConfigs: {pitch_code: {isLabel: true}}})
    .map(csvTransform)
    .shuffle(TRAINING_DATA_LENGTH)
    .batch(100);

// Load all training data in one batch to use for evaluation
const trainingValidationData =
    tf.data.csv(TRAIN_DATA_PATH, {columnConfigs: {pitch_code: {isLabel: true}}})
        .map(csvTransform)
        .batch(TRAINING_DATA_LENGTH);

// Load all test data in one batch to use for evaluation
const testValidationData =
    tf.data.csv(TEST_DATA_PATH, {columnConfigs: {pitch_code: {isLabel: true}}})
        .map(csvTransform)
        .batch(TEST_DATA_LENGTH);
```

5. 建立模型來分類投球類型

現在可以開始建構模型。使用 `tf.layers` API 將輸入內容 (形狀為 [8] 的音高感應器值) 連接至 3 個隱藏的全連接層 (由 ReLU 啟動單元組成)，然後連接至 1 個由 7 個單元組成的 Softmax 輸出層，每個單元代表一種輸出音高類型。

使用 adam 最佳化器和 `sparseCategoricalCrossentropy` 損失函式訓練模型。如要進一步瞭解這些選項，請參閱[訓練模型指南](#)。

在 `pitch_type.js` 結尾新增下列程式碼：

```
const model = tf.sequential();
model.add(tf.layers.dense({units: 250, activation: 'relu', inputShape: [8]}));
model.add(tf.layers.dense({units: 175, activation: 'relu'}));
model.add(tf.layers.dense({units: 150, activation: 'relu'}));
model.add(tf.layers.dense({units: NUM_PITCH_CLASSES, activation: 'softmax'}));

model.compile({
  optimizer: tf.train.adam(),
  loss: 'sparseCategoricalCrossentropy',
  metrics: ['accuracy']
});
```

稍後您會編寫主要伺服器程式碼，並從中觸發訓練。

如要完成 `pitch_type.js` 模組，請編寫函式來評估驗證和測試資料集、預測單一樣本的投球類型，以及計算準確度指標。將這段程式碼附加至 `pitch_type.js` 結尾：

```

// Returns pitch class evaluation percentages for training data
// with an option to include test data
async function evaluate(useTestData) {
  let results = {};
  await trainingValidationData.forEachAsync(pitchTypeBatch => {
    const values = model.predict(pitchTypeBatch.xs).dataSync();
    const classSize = TRAINING_DATA_LENGTH / NUM_PITCH_CLASSES;
    for (let i = 0; i < NUM_PITCH_CLASSES; i++) {
      results[pitchFromClassNum(i)] = {
        training: calcPitchClassEval(i, classSize, values)
      };
    }
  });

  if (useTestData) {
    await testValidationData.forEachAsync(pitchTypeBatch => {
      const values = model.predict(pitchTypeBatch.xs).dataSync();
      const classSize = TEST_DATA_LENGTH / NUM_PITCH_CLASSES;
      for (let i = 0; i < NUM_PITCH_CLASSES; i++) {
        results[pitchFromClassNum(i)].validation =
          calcPitchClassEval(i, classSize, values);
      }
    });
  }
  return results;
}

async function predictSample(sample) {
  let result = model.predict(tf.tensor(sample, [1, sample.length])).arraySync();
  var maxValue = 0;
  var predictedPitch = 7;
  for (var i = 0; i < NUM_PITCH_CLASSES; i++) {
    if (result[0][i] > maxValue) {
      predictedPitch = i;
      maxValue = result[0][i];
    }
  }
  return pitchFromClassNum(predictedPitch);
}

// Determines accuracy evaluation for a given pitch class by index
function calcPitchClassEval(pitchIndex, classSize, values) {
  // Output has 7 different class values for each pitch, offset based on
  // which pitch class (ordered by i)
  let index = (pitchIndex * classSize * NUM_PITCH_CLASSES) + pitchIndex;
  let total = 0;
  for (let i = 0; i < classSize; i++) {
    total += values[index];
    index += NUM_PITCH_CLASSES;
  }
  return total / classSize;
}

```

```
// Returns the string value for Baseball pitch labels
function pitchFromClassNum(classNum) {
  switch (classNum) {
    case 0:
      return 'Fastball (2-seam)';
    case 1:
      return 'Fastball (4-seam)';
    case 2:
      return 'Fastball (sinker)';
    case 3:
      return 'Fastball (cutter)';
    case 4:
      return 'Slider';
    case 5:
      return 'Changeup';
    case 6:
      return 'Curveball';
    default:
      return 'Unknown';
  }
}

module.exports = {
  evaluate,
  model,
  pitchFromClassNum,
  predictSample,
  testValidationData,
  trainingData,
  TEST_DATA_LENGTH
}
```

步驟 6 · 約 10 分鐘

6. 在伺服器上訓練模型

在名為 `server.js` 的新檔案中，編寫伺服器程式碼來執行模型訓練和評估。首先，請建立 HTTP 伺服器，並使用 `socket.io` API 開啟雙向通訊端連線。接著使用 `model.fitDataset` API 執行模型訓練，並使用您先前編寫的 `pitch_type.evaluate()` 方法評估模型準確率。訓練及評估 10 次疊代，並將指標列印至控制台。

將下列程式碼複製到 `server.js`：

```

require('@tensorflow/tfjs-node');

const http = require('http');
const socketio = require('socket.io');
const pitch_type = require('./pitch_type');

const TIMEOUT_BETWEEN_EPOCHS_MS = 500;
const PORT = 8001;

// util function to sleep for a given ms
function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// Main function to start server, perform model training, and emit stats via the socket
connection
async function run() {
  const port = process.env.PORT || PORT;
  const server = http.createServer();
  const io = socketio(server);

  server.listen(port, () => {
    console.log(` > Running socket on port: ${port}`);
  });

  io.on('connection', (socket) => {
    socket.on('predictSample', async (sample) => {
      io.emit('predictResult', await pitch_type.predictSample(sample));
    });
  });

  let numTrainingIterations = 10;
  for (var i = 0; i < numTrainingIterations; i++) {
    console.log(`Training iteration : ${i+1} / ${numTrainingIterations}`);
    await pitch_type.model.fitDataset(pitch_type.trainingData, {epochs: 1});
    console.log('accuracyPerClass', await pitch_type.evaluate(true));
    await sleep(TIMEOUT_BETWEEN_EPOCHS_MS);
  }

  io.emit('trainingComplete', true);
}

run();

```

此時，您已準備好執行及測試伺服器！您應該會看到類似下方的內容，伺服器會在每次疊代中訓練一個週期 (您也可以使用 `model.fitDataset` API，透過一次呼叫訓練多個週期)。如果此時發生任何錯誤，請檢查節點和 npm 安裝作業。

```
$ npm run start-server
...
  > Running socket on port: 8001
Epoch 1 / 1
eta=0.0
=====
=====>
2432ms 34741us/step - acc=0.429 loss=1.49
```

輸入 Ctrl-C 即可停止執行中的伺服器。我們會在下一個步驟再次執行。

7. 建立用戶端網頁並顯示程式碼

伺服器準備就緒後，下一步就是編寫在瀏覽器中執行的用戶端程式碼。建立簡單的網頁，在伺服器上叫用模型預測功能，並顯示結果。這項功能會使用 `socket.io` 進行用戶端/伺服器通訊。

首先，請在 `baseball/` 資料夾中建立 `index.html`：

```
<!doctype html>
<html>
  <head>
    <title>Pitch Training Accuracy</title>
  </head>
  <body>
    <h3 id="waiting-msg">Waiting for server...</h3>
    <p>
      <span style="font-size:16px" id="trainingStatus"></span>
    <p>
      <div id="predictContainer" style="font-size:16px;display:none">
        Sensor data: <span id="predictSample"></span>
        <button style="font-size:18px;padding:5px;margin-right:10px" id="predict-
button">Predict Pitch</button><p>
          Predicted Pitch Type: <span style="font-weight:bold" id="predictResult"></span>
        </div>
      <script src="dist/bundle.js"></script>
      <style>
        html,
        body {
          font-family: Roboto, sans-serif;
          color: #5f6368;
        }
        body {
          background-color: rgb(248, 249, 250);
        }
      </style>
    </body>
  </html>
```

接著，在 `baseball/` 資料夾中建立新的 `client.js` 檔案，並加入下列程式碼：

```

import io from 'socket.io-client';
const predictContainer = document.getElementById('predictContainer');
const predictButton = document.getElementById('predict-button');

const socket =
  io('http://localhost:8001',
    {reconnectionDelay: 300, reconnectionDelayMax: 300});

const testSample = [2.668,-114.333,-1.908,4.786,25.707,-45.21,78,0]; // Curveball

predictButton.onclick = () => {
  predictButton.disabled = true;
  socket.emit('predictSample', testSample);
};

// functions to handle socket events
socket.on('connect', () => {
  document.getElementById('waiting-msg').style.display = 'none';
  document.getElementById('trainingStatus').innerHTML = 'Training in Progress';
});

socket.on('trainingComplete', () => {
  document.getElementById('trainingStatus').innerHTML = 'Training Complete';
  document.getElementById('predictSample').innerHTML = '[' + testSample.join(', ') + ']';
  predictContainer.style.display = 'block';
});

socket.on('predictResult', (result) => {
  plotPredictResult(result);
});

socket.on('disconnect', () => {
  document.getElementById('trainingStatus').innerHTML = '';
  predictContainer.style.display = 'none';
  document.getElementById('waiting-msg').style.display = 'block';
});

function plotPredictResult(result) {
  predictButton.disabled = false;
  document.getElementById('predictResult').innerHTML = result;
  console.log(result);
}

```

用戶端會處理 `trainingComplete` 通訊端訊息，顯示預測按鈕。點選這個按鈕後，用戶端會傳送含有感應器資料樣本的通訊端訊息。收到 `predictResult` 訊息後，頁面會顯示預測結果。

步驟 8 · 約 5 分鐘

8. 執行應用程式

執行伺服器 and 用戶端，查看完整應用程式的運作情形：

```
[In one terminal, run this first]
$ npm run start-client

[In another terminal, run this next]
$ npm run start-server
```

在瀏覽器中開啟用戶端頁面 (<http://localhost:8080>)。模型訓練完成後，按一下「Predict Sample」(預測樣本) 按鈕。瀏覽器中應該會顯示預測結果。歡迎使用測試 CSV 檔案中的一些範例修改感應器資料樣本，看看模型預測的準確度。

步驟 9 · 約 5 分鐘

9. 您學到的內容

在本程式碼實驗室中，您使用 TensorFlow.js 實作了簡單的機器學習網頁應用程式。您已訓練自訂模型，根據感應器資料分類棒球投球類型。您編寫了 Node.js 程式碼，在伺服器上執行訓練，並使用從用戶端傳送的資料，對訓練好的模型呼叫推論。

請務必前往 tensorflow.org/js，查看更多範例和示範，以及相關程式碼，瞭解如何在應用程式中使用 TensorFlow.js。
